

Regression-Test Generation and Independent Oracles

T. R. Bentley | Verification Engineering Paper | September 2026

A repair and a regression test produced by the same model response are correlated evidence. The test may encode the same misunderstanding, repeat only the visible case, weaken expected behavior, or ratify its own patch.

This paper separates test generation from verification authority through provenance classes, admission gates, mutation challenges, hidden checks, and human review.

The attached datasets and diagrams are design specifications, not measured test-generation performance.

Key points

- Principle: a repair may propose its witness, but cannot appoint that witness as judge.
- Package: Markdown, PDF, two CSV matrices, and three SVG diagrams.

1. Repository baseline and evidence gap

AI Debugger Pro currently accepts a structured regression-test suggestion, displays it, and lets the user save it. Approved source is rerun and recorded as a verified or failed repair.

Repository tests are not automatically discovered or executed during approval. One rerun and one generated check therefore cannot establish semantic correctness, regression freedom, security, or generality.

Key points

- Preserve the human approval gate.
- Scope every verification label.
- Add independent evidence before stronger claims.

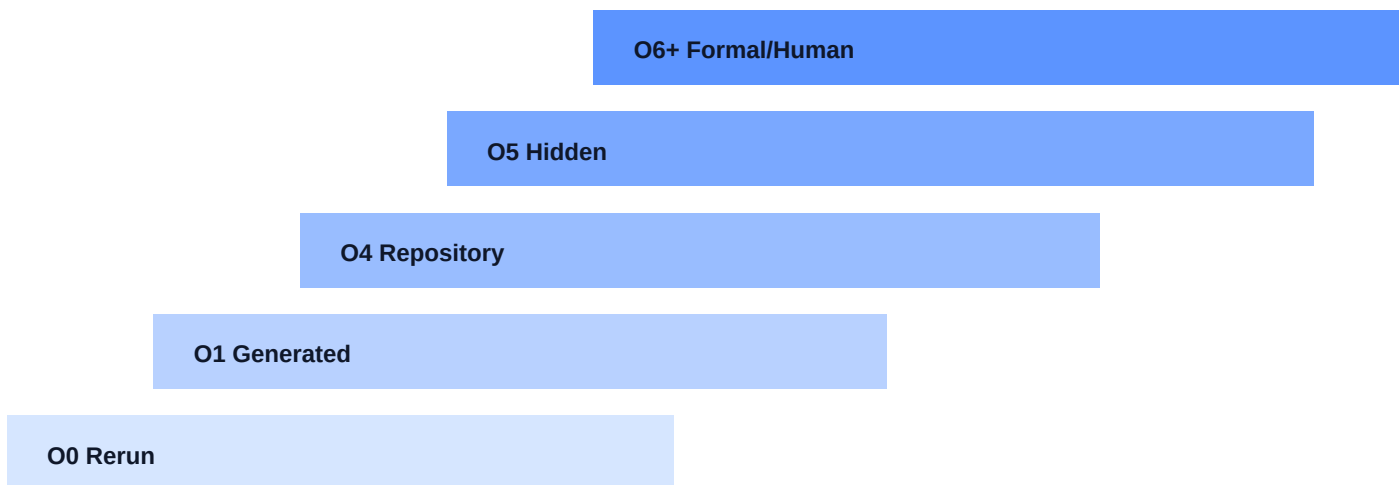
2. Oracle provenance hierarchy

Oracle authority depends on origin, repair visibility, requirement traceability, and independence. Candidate reruns and same-response tests provide weak evidence. Repository-owned, hidden, property-based, formal, security, and adjudicated oracles can support stronger claims when assumptions are documented.

Independence is graded rather than binary. Two artifacts can be separate files while sharing the same flawed premise.

Key points

- Record author, creation time, source requirement, context overlap, visibility, and modification history.
- Generated tests remain visibly generated after human approval.



3. Immutable test artifact contract

Each test artifact carries a stable ID, language and framework, oracle class, requirement reference, preconditions, fixtures, expected behavior, failure target, source and proposal hashes, generator, context hash, mutation evidence, outcomes, and admission state.

Editing a test creates a new version and invalidates prior evidence. A candidate cannot rewrite the check that rejected it.

Key points

- Bind tests to exact source and proposal states.
- Separate raw execution evidence from policy decisions.
- Keep provenance queryable in history.

4. Admission pipeline

The six gates parse and isolate the test, bind it to independent intent, require fail-before and pass-after behavior, challenge plausible wrong repairs, check noninterference, and admit the result with scoped authority.

Possible outcomes are reject, revise, quarantine, admit, or request clarification.

Key points

- A test that already passes on the known-bad version does not reproduce the defect.
- A test that skips or weakens a stronger oracle is blocked.
- Green is an observation, not a coronation.

Parse -> Bind intent -> Fail bad -> Pass candidate -> Challenge -> Admit

Every gate records evidence and may reject, revise, quarantine, or escalate.

5. Failure modes

Common failures include visible-example duplication, patch mirroring, constant-answer acceptance, exception suppression, test weakening, over-mocking, environmental dependence, security blindness, flakiness, stale binding, vacuous properties, and oracle conflict.

Each failure needs a specific challenge rather than a generic pass/fail check.

Key points

- Boundary partitions challenge overfit.
- Semantic assertions challenge suppression.
- Policy oracles challenge dangerous side effects.
- Mutation checks expose vacuous tests.

6. Mutation and independent strategies

Mutation testing evaluates whether a test distinguishes intended behavior from declared plausible errors. Operators include condition inversions, boundary shifts, constant returns, removed exception paths, skipped branches, altered loop limits, and removed security checks.

Strong strategies include repository suites, specification-derived examples, property and metamorphic checks, differential references, formal contracts, security policies, and human adjudication.

Key points

- Mutation score is not proof of correctness.
- Surviving mutants reveal specific gaps.
- Common-mode assumptions must be documented.

7. Language-specific harnesses

Python should preserve pytest outcomes, assertion diffs, exception chains, fixture provenance, and seeds. C and C++ require compiler profiles, return codes, sanitizer evidence, and explicit handling of silent exits. Java requires JUnit outcomes, exception chains, classpath, packages, and build metadata.

A shared result schema records build, discovery, execution, assertion, timeout, policy, and artifact evidence while preserving language semantics.

Key points

- Do not flatten compile and assertion failures into one bucket.
- Record framework and toolchain versions.
- Keep sandbox and side-effect evidence separate.

8. Frozen scenarios and evaluation

Sixteen frozen scenarios specify expected admission outcomes for correct reproduction, overfit, suppression, weakening, skipped tests, mocks, environment dependence, flakiness, stale bindings, security effects, vacuity, properties, differential oracles, and ambiguous intent.

Future measurements include reproduction rate, mutation score, overfit and suppression detection, security detection, flake rate, leakage, admission precision, and admission recall.

Key points

- Generated tests never label themselves.
- Model comparisons require hidden independent oracles.
- Ambiguous intent requires adjudication, not invented certainty.

9. Human authority, roadmap, and conclusion

The review surface should show origin, requirement link, test diff, affected production code, bad-version failure, candidate pass, mutation challenges, side effects, and conflicts. Approval changes governance state but never erases provenance.

Implementation proceeds through provenance storage, test discovery and language execution, admission challenges, and interface plus preregistered evaluation.

Tests become accountable witnesses when provenance, challenge power, and authority remain separate.

Key points

- Paper 10 -> Paper 11 navigation is included.
- Final planned paper: Reproducible Engineering and Release Evidence.
- A generated test can help; it cannot certify its own repair.